ANÁLISIS DE ALGORITMOS

Apuntes de estudio y consulta para examen de programación

Complejidad · Logaritmos · Teorema Maestro · Optimización



Guía compacta: comprensión · resolución paso a paso · consulta rápida

Índice

1. Fundamentos de complejidad 3

2. Tabla de complejidades 4

3. Cómo sacar la complejidad de un código 5

4. Logaritmos: log n, log log n y n log n 7

5. Complejidad espacial 9

6. Teorema Maestro 10

7. Cómo mejorar algoritmos ineficientes 12

8. Hoja de consulta rápida para el examen 15

Cómo usar estos apuntes

1. Lee las secciones 1–7 una vez para entender; los ejemplos van separados de la teoría y en recuadros. **2.** Practica el método de la sección 3 y la plantilla del Teorema Maestro (sección 6). **3.** En el examen, usa la **sección 8** (3 páginas, a doble columna): es una hoja independiente.

Convenciones: log = $\log_2$ salvo indicación; la base no cambia la clase de complejidad. Código en Python. Todas las complejidades se dan como Θ/O del peor caso salvo que se indique lo contrario.

Recuadros de colores
Azul-verde: regla / idea clave · **Naranja**: cuidado / error frecuente · **Verde**: ejemplo / plantilla · **Azul oscuro**: fórmula

1. Fundamentos de complejidad

La **complejidad algorítmica** mide cuántos recursos (tiempo o memoria) necesita un algoritmo en función del tamaño de la entrada, sin depender de la máquina ni del lenguaje. Se expresa con **notación asintótica**: cómo crece el costo cuando $n \to \infty$.

Concepto	Qué es
n	Tamaño de la entrada: nº de elementos de un arreglo, longitud de una cadena, nº de nodos. Si la entrada es un número N, aclara si n = N (valor) o n = log N (bits).
Complejidad temporal T(n)	Nº de *operaciones elementales* (comparar, asignar, sumar, acceder a a[i]), no segundos.
Complejidad espacial S(n)	Memoria que usa el algoritmo en función de n (ver sección 5).
Operación elemental	Cuesta O(1): aritmética, comparación, asignación, acceso por índice.

1.1 Big O, Omega y Theta

Notación	Significa	Definición formal ($\exists\, c, n_0 > 0, \forall\, n \geq n_0$)	Se lee
O(g)	Cota superior	$f(n) \leq c \cdot g(n)$	f no crece más rápido que g
Ω(g)	Cota inferior	$f(n) \geq c \cdot g(n)$	f no crece más lento que g
Θ(g)	Cota ajustada	$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ (O y Ω a la vez)	f crece igual que g

Idea clave

$3n^2 + 5n + 2 \in \Theta(n^2)$. También es $O(n^3)$ (cota válida pero no ajustada) y $\Omega(n)$. En un examen, "la complejidad" suele significar la cota **Θ (o O) más ajustada del peor caso**.

O no significa "peor caso". O, Ω y Θ son cotas de una función; mejor/promedio/peor son escenarios de entrada. Cada escenario tiene su propia función y por tanto sus propias cotas.

1.2 Mejor caso, caso promedio y peor caso

Algoritmo	Mejor caso	Promedio	Peor caso	Cuándo ocurre el mejor / peor
Búsqueda lineal	O(1)	O(n)	O(n)	Está en la 1ª posición / no está
Búsqueda binaria	O(1)	O(log n)	O(log n)	Está en el medio / no está
Insertion sort	O(n)	$O(n^2)$	$O(n^2)$	Ya ordenado / orden inverso
Merge sort	O(n log n)	O(n log n)	O(n log n)	Siempre igual (versión estándar)
Quick sort (pivote simple)	O(n log n)	O(n log n)	$O(n^2)$	Pivote parte a la mitad / pivote siempre extremo
Tabla hash (buscar)	O(1)	O(1)	O(n)	Sin colisiones / todas las claves colisionan

1.3 Cómo simplificar expresiones

Se ELIMINA	Ejemplo	Se CONSERVA	Ejemplo
Constantes multiplicativas	$5n^2 \to n^2$	El término de mayor crecimiento	$n^2 + 100n + 7 \to n^2$
Constantes aditivas	$n + 500 \to n$	Potencias y exponentes	$n^2 \neq n^3$; $2^n \neq 3^n$
Términos de menor orden	$n^3 + n^2 \to n^3$	El log que multiplica	$n \log n \neq n$
Base del logaritmo	$\log_{10} n = \Theta(\log_2 n)$	Variables independientes distintas	O(n + m), O(n·m)
Coeficiente / exponente dentro del log	$\log(n^2) = 2 \log n \to \log n$	Exponente de un log	$(\log n)^2 \neq \log n$

$$1 < \log\log n < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < 2^n < n! < n^n$$

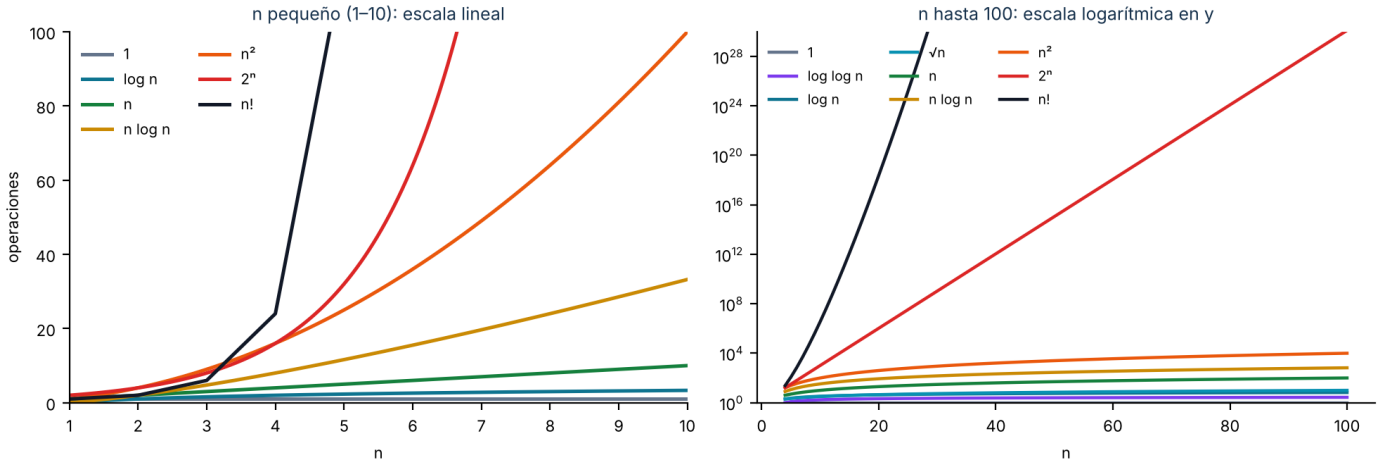
Orden de crecimiento (de menor a mayor). Cualquier polinomio en log n crece más lento que cualquier n^ε, $\varepsilon > 0$; cualquier polinomio crece más lento que 2^n.

1.4 Tabla de reglas rápidas

Regla	Fórmula	Ejemplo
Secuencia (suma)	$O(f) + O(g) = O(\max(f, g))$	$O(n) + O(n^2) = O(n^2)$
Anidación (producto)	$O(f) \cdot O(g) = O(f \cdot g)$	ciclo n dentro de ciclo n → $O(n^2)$
Constante	$O(c \cdot f) = O(f)$	$O(7n) = O(n)$
Polinomio	$a_k n^k + \ldots + a_0 = \Theta(n^k)$	$4n^3 + 9n + 1 = \Theta(n^3)$
Suma 1 + 2 + ... + n	$n(n+1)/2 = \Theta(n^2)$	ciclos con $j < i$ → $O(n^2)$
Suma geométrica	$1 + 2 + 4 + \ldots + 2^k = 2^{k+1} - 1 = \Theta(\text{último término})$	ciclo i *= 2 con trabajo i → O(n)
Suma armónica	$1 + 1/2 + \ldots + 1/n \approx \ln n$	$\Sigma\, n/i = n \cdot H_n \to O(n \log n)$
Exponencial vs polinomio	$n^k = O(2^n)$ pero $2^n \neq O(n^k)$	$n^{1000} \in O(2^n)$
Base de exponencial	$2^{n+1} = \Theta(2^n)$ pero $2^{2n} = 4^n \neq \Theta(2^n)$	3^n no es $O(2^n)$
Variables distintas	No se pueden fusionar	O(n + m), O(n·m), O(n log m)

2. Tabla de complejidades

Complejidad	Nombre	Patrón típico	Ejemplo	Ops. n = 10^3
O(1)	Constante	Acceso directo	array[i], push en pila	1
O(log n)	Logarítmica	División o multiplicación constante	Búsqueda binaria; i *= 2	≈ 10
O(log log n)	Log logarítmica	Reducción exponencial del tamaño (n → √n, i → i^2)	i = i*i; interpolation search (datos uniformes)	≈ 3
O(√n)	Raíz cuadrada	Incrementos hasta √n	while i*i <= n; primalidad por divisiones	≈ 32
O(n)	Lineal	Un recorrido	Buscar un elemento; sumar un arreglo	10^3
O(n log n)	Lineal-logarítmica	Dividir y recorrer; n × log n	Merge Sort, Heap Sort	≈ 10^4
O(n^2)	Cuadrática	Dos ciclos dependientes	Comparaciones por pares; burbuja	10^6
O(n^3)	Cúbica	Tres ciclos anidados	Multiplicación ingenua de matrices	10^9
O(2^n)	Exponencial	Ramificación recursiva	Fibonacci ingenuo; subconjuntos	≈ 10^{301}
O(n!)	Factorial	Permutaciones	Generar permutaciones; TSP por fuerza bruta	≈ 10^{2567}



Izquierda: con n pequeño 2^n y n! ya se salen de la gráfica. Derecha: en escala logarítmica se ve la jerarquía completa; log log n es casi una recta plana.

2.1 Diferenciar log n, log log n y n log n

n	$2^4 = 16$	$2^8 = 256$	$2^{16} = 65\,536$	$2^{32} \approx 4.3 \cdot 10^9$
$\log_2 n$	4	8	16	32
$\log_2 \log_2 n$	2	3	4	5
n $\log_2 n$	64	2 048	1 048 576	≈ $1.4 \cdot 10^{11}$

	O(log n)	O(log log n)	O(n log n)
Señal en el código	La variable se **multiplica o divide** por una constante (i *= 2, n //= 3)	La variable se **eleva al cuadrado** o se le saca raíz (i = i*i, n = √n)	Ciclo de n con un log dentro, o dividir a la mitad y recorrer todo n por nivel
Intuición	Cuántas veces divido n entre 2 hasta llegar a 1	Cuántas veces divido el *número de bits* de n entre 2	n trabajo × log n niveles
Error típico	Confundirlo con n/2 (eso es lineal)	Confundirlo con log n o con $(\log n)^2$	Confundirlo con n^2 o con n

3. Cómo sacar la complejidad de un código

Método en 4 pasos

1) Identifica n. **2)** Cuenta cuántas veces se ejecuta cada ciclo (escribe los valores de la variable y despeja k). **3)** Ciclos anidados → **multiplica**; bloques seguidos → **suma** (gana el mayor). **4)** Simplifica (sección 1.3). En recursión: plantea T(n) y resuélvela (sección 6).

3.1 Ciclo simple — O(n)

```
for i in range(n):
    x += 1

# variante: paso constante
for i in range(0, n, 3):
    x += 1
```

El cuerpo se ejecuta n veces y cada vez cuesta O(1): total = n · 1 = n.
Con paso c: n/c iteraciones, y n/c sigue siendo O(n).

Regla rápida: la variable avanza de a una constante (i += c) hasta n → lineal.

3.2 Dos ciclos anidados — $O(n^2)$

```
for i in range(n):
    for j in range(n):
        x += 1
```

El interno hace n iteraciones por cada una de las n del externo: n · n = n^2.
Con límites distintos (n y m): O(n·m). Tres ciclos: $O(n^3)$.

Regla rápida: ciclos anidados independientes → se multiplican.

3.3 Ciclos consecutivos — $O(n^2)$

```
for i in range(n):          # n
    a += 1
for i in range(n):          # n²
    for j in range(n):
        b += 1
```

Primer bloque: n. Segundo bloque: n^2. Total: $n + n^2$.
Se conserva el término dominante: n^2 (secuencia = suma = máximo).

Regla rápida: bloques seguidos → se suman → gana el de mayor orden.

3.4 Ciclo con i *= 2 — O(log n)

```
i = 1
while i < n:
    i *= 2
```

Valores de i: 1, 2, 4, 8, ..., 2^k. Se detiene cuando $2^k \geq n$.
Despeje: $k = \log_2 n$. Si fuera i *= 3, $k = \log_3 n$ (misma clase).
Requiere i inicial ≥ 1 (con i = 0 nunca crece).

Regla rápida: se multiplica por constante > 1 → logarítmico.

3.5 Ciclo con i /= 2 — O(log n)

```
i = n
while i >= 1:
    i //= 2
```

Valores: n, n/2, n/4, ..., $n/2^k$. Termina cuando $n/2^k < 1 \rightarrow 2^k > n$.
$k \approx \log_2 n + 1$ iteraciones.

Regla rápida: se divide entre constante → logarítmico (como la búsqueda binaria).

3.6 Ciclo con i *= i — O(log log n)

```
i = 2
while i < n:
    i = i * i
```

Valores: 2, 2^2, 2^4, 2^8, ... = 2^{2^k}. Termina cuando $2^{2^k} \geq n$.
Tomando $\log_2$: $2^k \geq \log_2 n \rightarrow$ **$k = \log_2 \log_2 n$**.
Ojo: i inicial debe ser ≥ 2 (con 1 o 0 el ciclo es infinito).

Regla rápida: se eleva al cuadrado (o se saca raíz) → log log n.

3.7a Incrementos variables: √n — O(√n)

```
i, k = 0, 1
while i < n:
    i += k
    k += 1

# equivalente: tope √n
while i * i <= n:
    i += 1
```

i = 1 + 2 + ... + k = k(k+1)/2 ≥ n → k ≈ √(2n).
Segundo ciclo: $i^2 \le n \rightarrow i \le \sqrt{n}$. Ambos O(√n).

Regla rápida: el incremento crece de 1 en 1, o la condición es $i^2 \le n$ → raíz.

3.7b Incrementos variables: n log n — O(n log n)

```
for i in range(n):
    j = 1
    while j < n:
        j *= 2

# paso proporcional a i (armónica)
for i in range(1, n + 1):
    for j in range(0, n, i):
        x += 1
```

1er bloque: n iteraciones × log n del while = n log n.
2º bloque: el interno hace n/i iteraciones → $\Sigma\, n/i = n(1 + 1/2 + ... + 1/n) = n \cdot H_n \approx n \ln n$.

Regla rápida: ciclo de n con log dentro, o paso j += i con i variable → n log n.

3.8a Anidados dependientes — O(n²)

```
for i in range(n):
    for j in range(i):
        x += 1
```

El interno hace i iteraciones: 0 + 1 + ... + (n−1) = n(n−1)/2.
Constante 1/2 y término lineal se eliminan. Si j llega a i^2: $\Sigma\, i^2 \approx n^3/3 \rightarrow O(n^3)$.

Regla rápida: el interno depende de i y crece linealmente → suma aritmética → n^2.

3.8b Dependiente con suma geométrica — O(n)

```
i = 1
while i < n:
    for j in range(i):
        x += 1
    i *= 2
```

Trabajo: $1 + 2 + 4 + ... + 2^k$ con $2^k < n$.
Suma geométrica = $2^{k+1} - 1 < 2n \rightarrow$ **O(n)**, no O(n log n).

Regla rápida: si el trabajo crece geométricamente, domina el último término.

3.9 Recursividad simple — O(n)

```
def f(n):
    if n <= 1: return
    f(n - 1)
    x += 1      # O(1) por llamada
```

T(n) = T(n−1) + 1 → n llamadas × O(1) = n.
Variantes: T(n−1) + n → n(n+1)/2 → $O(n^2)$; T(n/2) + 1 → O(log n); T(n/2) + n → n + n/2 + ... = 2n → O(n).

Regla rápida: nº de llamadas × trabajo por llamada. Reduce por resta → cuenta n; por división → log n.

3.10 Recursividad múltiple — O(2ⁿ)

```
def g(n):
    if n <= 1: return 1
    return g(n-1) + g(n-1)

def fib(n):
    if n <= 1: return n
    return fib(n-1) + fib(n-2)
```

g: T(n) = 2T(n−1) + 1 → árbol de altura n y 2 ramas: $1 + 2 + ... + 2^n = 2^{n+1} - 1 \rightarrow O(2^n)$.
fib: $T(n) = T(n-1) + T(n-2) + 1 = \Theta(\varphi^n)$, $\varphi \approx 1.618$, acotado por $O(2^n)$.
Con división: 2T(n/2) + n → O(n log n); $3T(n/2) + n \rightarrow O(n^{\log_2 3})$ (Maestro).

Regla rápida: $\text{ramas}^{\text{profundidad}}$. Reduce por resta con 2+ llamadas → exponencial.

3.11 Condicionales — O(n) peor

```
if x > 0:
    for i in range(n):   # O(n)
        s += 1
else:
    s = 0                # O(1)
```

Mejor caso: rama else, O(1). Peor caso: rama if, O(n).
Costo = costo de la condición + máx(ramas). Un break/return temprano mejora el mejor caso, no el peor.

Regla rápida: analiza cada rama y toma la mayor (peor caso); suma el costo de evaluar la condición.

3.12 Código mixto — $O(n^2)$

```python
def mixto(n):
    for i in range(n):          # n
        j = 1
        while j < n:            # log n
            j *= 2
    for i in range(n // 2):     # n/2
        for j in range(n):      # n
            x += 1
    k = n
    while k > 1:                # log n
        k //= 2
```

Bloque 1: $n \cdot \log n$.
Bloque 2: $(n/2) \cdot n = n^2/2$.
Bloque 3: $\log n$.
Suma: $n \log n + n^2/2 + \log n \to$ domina n^2.

Regla rápida: calcula cada bloque por separado, suma y conserva el mayor.

3.13 Tabla final: patrón de código → complejidad

Patrón	Iteraciones / razonamiento	Complejidad
for i < n: i++ | i += c	n | n/c	**O(n)**
while i < n: i *= c (c ≥ 2)	$c^k = n \to k = \log_c n$	**O(log n)**
while i ≥ 1: i /= c	$n/c^k = 1 \to k = \log_c n$	**O(log n)**
while i < n: i = i * i ($i_0 \geq 2$)	$2^{2^k} = n \to k = \log \log n$	**O(log log n)**
while i*i ≤ n: i++ | i += k; k++	$\sqrt{n}$ | $k(k+1)/2 = n$	**O(√n)**
for n { for n }	$n \cdot n$	**O(n²)**
for i { for j < i }	$n(n-1)/2$	**O(n²)**
for n { while j < n: j *= 2 }	$n \cdot \log n$	**O(n log n)**
for i { for j in range(0, n, i) }	$\Sigma\, n/i = n \cdot H_n$	**O(n log n)**
while i < n: for j < i ...; i *= 2	suma geométrica ≈ 2n	**O(n)**
T(n) = T(n−1) + 1	n llamadas	**O(n)**
T(n) = T(n−1) + n	n + (n−1) + ... + 1	**O(n²)**
T(n) = T(n/2) + 1	log n niveles	**O(log n)**
T(n) = 2T(n/2) + n	n por nivel × log n niveles	**O(n log n)**
T(n) = 2T(n−1) + 1	$\text{ramas}^{\text{profundidad}}$	**O(2^n)**
if / else	costo de la condición + máx(ramas)	**peor caso = mayor rama**
bloque A; bloque B	O(A) + O(B)	**máx(A, B)**

4. Logaritmos: log n, log log n y n log n

4.1 Fórmulas que debes dominar

Logaritmos	Sumas útiles
$\log_b x = y \Leftrightarrow b^y = x$ (¿cuántas veces divido x entre b hasta llegar a 1?)	$\Sigma_{i=1}^{n}\, i = n(n+1)/2$
$\log_b(x \cdot y) = \log_b x + \log_b y$; $\log_b(x/y) = \log_b x - \log_b y$	$\Sigma_{i=0}^{k}\, 2^i = 2^{k+1} - 1$
$\log_b(x^k) = k \cdot \log_b x$; $\log_b 1 = 0$; $\log_b b = 1$	$\Sigma_{i=0}^{k}\, r^i = (r^{k+1} - 1)/(r - 1)$
Cambio de base: $\log_b x = \log_c x / \log_c b$ (solo cambia una constante)	$\Sigma_{i=1}^{n}\, 1/i \approx \ln n$ (armónica)
$b^{\log_b x} = x$; $a^{\log_b c} = c^{\log_b a}$ (útil en Teorema Maestro)	$\log(n!) = \Theta(n \log n)$
log log n = log(log n) → $n = 2^{32}$: log n = 32, log log n = 5	$\log_2 10^3 \approx 10$; $\log_2 10^6 \approx 20$; $\log_2 10^9 \approx 30$

4.2 O(log n)

Despeje matemático (3 pasos)
1) Escribe los valores sucesivos de la variable. **2)** Iguala el valor tras k iteraciones con la condición de salida. **3)** Despeja k.
i *= 3 con i < n: $3^k = n \to k = \log_3 n$. **i = n; i //= 2** hasta 1: $n/2^k = 1 \to k = \log_2 n$. La base no importa: $\Theta(\log n)$.

Búsqueda binaria

```python
def bsearch(a, x):           # a ordenado
    lo, hi = 0, len(a) - 1
    while lo <= hi:
        mid = (lo + hi) // 2
        if a[mid] == x: return mid
        elif a[mid] < x: lo = mid + 1   # descarta mitad izq.
        else: hi = mid - 1              # descarta mitad der.
    return -1
```

Cada iteración descarta la mitad: tamaño n, n/2, n/4, ..., 1 → **$\lfloor \log_2 n \rfloor + 1$** iteraciones. Recurrencia: T(n) = T(n/2) + O(1) = Θ(log n) (Maestro, caso 2). Requiere arreglo **ordenado** y acceso por índice O(1).

Dividir entre 10

```python
while n > 0:     # contar dígitos de n
    n //= 10     # log₁₀ n iteraciones
```

Exponenciación rápida: T(n) = T(n/2) + 1

```python
def pw(x, n):       # potencia rápida
    if n == 0: return 1
    h = pw(x, n // 2)  # UNA sola llamada
    return h*h if n % 2 == 0 else h*h*x
```

Trampa: si en pw se llamara dos veces a pw(x, n//2), T(n) = 2T(n/2) + 1 = Θ(n). Guarda el resultado en una variable.

4.3 O(log log n)

Qué significa: cuántas veces puedo sacar raíz cuadrada a n hasta llegar a ≤ 2 (o elevar al cuadrado desde 2 hasta n). En log n se reduce el *tamaño* a la mitad; en log log n se reduce a la mitad el *número de bits* (el exponente). Crece lentísimo: $n = 2^{64}$ → log log n = 6.

Cómo aparece: (a) i = i*i o i = i**2; (b) recursión n → √n: T(n) = T(√n) + 1; (c) interpolation search sobre datos uniformes (promedio); (d) árboles de van Emde Boas (O(log log U)); (e) la Criba de Eratóstenes es Θ(n log log n).

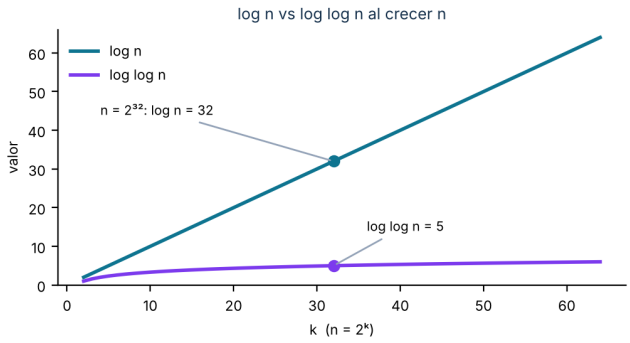
Reducción n → √n

```python
def r(n):
    if n <= 2: return
    r(int(n ** 0.5))     # T(n) = T(√n) + 1
```

Demostración (raíz): $n \to n^{1/2} \to n^{1/4} \to \dots \to n^{1/2^k}$. Se detiene cuando $n^{1/2^k} \le 2$. Tomando $\log_2$: $(1/2^k) \cdot \log_2 n \le 1 \to 2^k \ge \log_2 n \to$ **$k = \log_2 \log_2 n$**.
Demostración (cuadrado): $i_k = 2^{2^k}$ (2, 4, 16, 256, 65 536, ...). $2^{2^k} \ge n \to 2^k \ge \log_2 n \to k = \log_2 \log_2 n$.
Truco: sea m = log n. Entonces T(n) = T(√n) + 1 se vuelve S(m) = S(m/2) + 1 = log m = log log n.

Diferencia con log n

Con $n = 2^{32} = 4\,294\,967\,296 \approx 4.3 \cdot 10^9$: log n = 32 pasos, log log n = 5 pasos. Pasar de 2^{32} a 2^{64} (elevar n al cuadrado) duplica log n (32 → 64) pero solo suma 1 a log log n (5 → 6).



4.4 O(n log n)

Dos formas de aparecer: (A) un ciclo de n iteraciones con trabajo log n por iteración (ejemplo 3.7b); (B) **dividir y vencerás**: se parte el problema en mitades y en cada nivel del árbol se recorren los n elementos: T(n) = 2T(n/2) + n.

Merge Sort: 2T(n/2) + O(n)

```python
def merge_sort(a):
    if len(a) <= 1: return a
    m = len(a) // 2
    L = merge_sort(a[:m])
    R = merge_sort(a[m:])
    return merge(L, R)      # mezcla: O(n)
```

Nivel	Subproblemas	Tamaño c/u	Trabajo / nivel
0	1	n	n
1	2	n/2	n
2	4	n/4	n
k	2^k	$n/2^k$	n
$\log_2 n$	n	1	n

Árbol de recursión: la altura es $\log_2 n$ (se divide entre 2 hasta llegar a 1) y cada nivel suma n trabajo. Total = $n \cdot (\log_2 n + 1)$ = **Θ(n log n)**. Además, ordenar por comparaciones requiere Ω(n log n) porque log(n!) = Θ(n log n); por eso Merge Sort y Heap Sort

son óptimos.

4.5 Cuadro comparativo

	O(log n)	O(log log n)	O(n log n)
Qué ocurre	Tamaño ÷ constante	Tamaño al cuadrado / raíz	n trabajo × log n niveles
Patrón de código	i *= 2; i //= 2	i = i*i; n = √n	for + while *= 2; Merge Sort
Despeje	$c^k = n \to k = \log_c n$	$2^{2^k} = n \to k = \log \log n$	altura log n × n por nivel
Recurrencia	T(n) = T(n/2) + 1	T(n) = T(√n) + 1	T(n) = 2T(n/2) + n
Ejemplos	Búsqueda binaria, potencia rápida	Interpolation search, van Emde Boas	Merge Sort, Heap Sort
n = 2^{32}	32	5	≈ $1.4 \cdot 10^{11}$
Error común	Confundir con n/2	Confundir con log n o $(\log n)^2$	Confundir con n^2 o con n

5. Complejidad espacial

Mide la memoria que usa el algoritmo en función de n. Se distingue: **espacio total** = entrada + memoria extra; **espacio auxiliar** = solo la memoria extra (variables, estructuras temporales, pila de recursión). En exámenes casi siempre se pide el **auxiliar**; indica cuál usas.

Qué usa el algoritmo	Espacio	Comentario
Variables escalares (contadores, índices, sumas)	**O(1)**	No dependen de n
Arreglo / lista de n elementos	**O(n)**	Si es la entrada, no cuenta como auxiliar
Matriz n × m	**O(n·m)**	n × n → $O(n^2)$
Copia de arreglo, a[i:], concatenar listas	**O(n)** cada una	El slicing copia: dentro de un ciclo o recursión se acumula
Conjunto / diccionario con n elementos	**O(n)**	Hash tables: tiempo O(1) a cambio de memoria O(n)
Recursión de profundidad d	**O(d)**	Pila = profundidad máxima × tamaño del marco
Memoización de n estados	**O(n)**	Más la pila si es top-down
Tabla de programación dinámica n × m	**O(n·m)**	Se reduce a O(m) si solo necesitas la fila anterior

5.1 Recursividad y pila de llamadas

Idea clave

El espacio de pila es la **profundidad máxima** de la recursión (cuántos marcos están activos a la vez), **no** el número total de llamadas. Fibonacci ingenuo hace $\Theta(\varphi^n)$ llamadas pero su profundidad es n → espacio O(n), no $O(2^n)$.

Iterativa

```
def suma_it(a):          # espacio O(1)
    s = 0
    for x in a:
        s += x
    return s
```

Recursiva (n marcos en la pila)

```
def suma_rec(a, i=0):    # espacio O(n)
    if i == len(a): return 0
    return a[i] + suma_rec(a, i + 1)

# TRAMPA: suma_rec(a[1:]) copia en cada nivel
# → tiempo O(n²) y espacio O(n²)
```

5.2 Tabla de consulta: tiempo y espacio auxiliar

Algoritmo	Tiempo	Espacio auxiliar	Por qué
Burbuja / Selección / Inserción	$O(n^2)$	$O(1)$	Ordenan en el lugar
Merge Sort	$O(n \log n)$	$O(n)$	Arreglo auxiliar para mezclar (+ pila $O(\log n)$)
Quick Sort	$O(n \log n)$ prom., $O(n^2)$ peor	$O(\log n)$ prom., $O(n)$ peor	Solo la pila de recursión
Heap Sort	$O(n \log n)$	$O(1)$	En el lugar, sin recursión
Búsqueda binaria iterativa / recursiva	$O(\log n)$	$O(1)$ / $O(\log n)$	La recursiva apila log n marcos
Factorial recursivo	$O(n)$	$O(n)$	n marcos en la pila
Fibonacci ingenuo	$\Theta(\varphi^n) \subset O(2^n)$	$O(n)$	Profundidad máxima n
Fibonacci con memoización	$O(n)$	$O(n)$	Memo $O(n)$ + pila $O(n)$
Fibonacci iterativo (2 variables)	$O(n)$	$O(1)$	Solo guarda los dos últimos
DFS / BFS (grafo)	$O(V + E)$	$O(V)$	Visitados + pila / cola

Compromiso tiempo ↔ espacio: memoizar y usar hash tables gastan memoria para ahorrar tiempo; Heap Sort ahorra memoria frente a Merge Sort.

6. Teorema Maestro

$T(n) = a \cdot T(n/b) + f(n)$ con $a \geq 1$, $b > 1$, $f(n)$ asintóticamente positiva

Símbolo	Significado	Dónde se ve
a	Cantidad de subproblemas (llamadas recursivas)	nº de llamadas a f(...) en el código
b	Factor de reducción: cada subproblema es n/b	se divide entre b (mitad → b = 2)
n/b	Tamaño de cada subproblema	argumento de la llamada
f(n)	Trabajo fuera de las llamadas (dividir + combinar)	ciclos / mezcla dentro de la función
$p = \log_b a$	Exponente de comparación; n^p = nº de hojas del árbol de recursión	se compara con f(n)
k	Exponente del logaritmo en $f(n) = n^p \log^k n$ (solo caso 2; $k \geq 0$)	ej.: $f = n \log n \rightarrow k = 1$

Intuición: compara el trabajo en las **hojas** (n^p) con el trabajo en la **raíz** (f(n)). Domina el mayor; si empatan, todos los niveles aportan igual y se multiplica por log n.

Caso	Condición sobre f(n)	Resultado	Quién domina
1	$f(n) = O(n^{p-\varepsilon})$, $\varepsilon > 0$ (f crece polinomialmente *más lento* que n^p)	$T(n) = \Theta(n^p)$	Las hojas
2	$f(n) = \Theta(n^p \log^k n)$, $k \geq 0$ (f crece *igual* que n^p)	$T(n) = \Theta(n^p \log^{k+1} n)$	Todos los niveles por igual
3	$f(n) = \Omega(n^{p+\varepsilon})$, $\varepsilon > 0$ y **$a \cdot f(n/b) \leq c \cdot f(n)$**, $c < 1$ (regularidad)	$T(n) = \Theta(f(n))$	La raíz

Atajo cuando $f(n) = n^d$ (o $n^d \log^k n$)
Compara **a** con $\mathbf{b^d}$ (equivale a comparar p con d): $a > b^d \rightarrow$ **Caso 1** → $\Theta(n^p)$ | $a = b^d \rightarrow$ **Caso 2** → $\Theta(n^d \log^{k+1} n)$ | $a < b^d \rightarrow$ **Caso 3** → $\Theta(n^d \log^k n)$.
Para f polinomial (con o sin log) la condición de regularidad se cumple sola.

Plantilla de resolución para el examen
1. Identifica a. **2.** Identifica b. **3.** Identifica f(n). **4.** Calcula $p = \log_b a$. **5.** Compara f(n) con n^p (¿menor, igual o mayor polinomialmente?). **6.** Elige el caso (en el 3, verifica la regularidad). **7.** Escribe $\Theta(...)$.

6.1 Los tres casos con ejemplo resuelto

CASO 1 — gana el trabajo de las hojas	
Cuándo se usa	Muchos subproblemas y poco trabajo por llamada: f(n) es polinomialmente menor que n^p.
Cómo reconocerlo	$a > b^d$. Ejemplo típico: f(n) = n o n^2 con a grande.
Ejemplo	$T(n) = 8T(n/2) + n^2$ (multiplicación ingenua de matrices por bloques).
Pasos	a = 8, b = 2, $f(n) = n^2$. $p = \log_2 8 = 3$. Comparar: $n^2 = O(n^{3-1})$ → ε = 1 > 0 → **Caso 1**. Resultado: **$T(n) = \Theta(n^3)$**.

CASO 2 — empate: todos los niveles pesan igual	
Cuándo se usa	El trabajo por nivel es el mismo en todos los niveles; se acumula log n niveles (más $\log^k$ si f lleva log).
Cómo reconocerlo	$a = b^d$. $f(n) = n^p$ exacto (k = 0), o n^p multiplicado por $\log^k n$.
Ejemplo	$T(n) = 2T(n/2) + n$ (Merge Sort) y $T(n) = 2T(n/2) + n \log n$.
Pasos	a = 2, b = 2, $p = \log_2 2 = 1$. (i) $f = n = n^1 \log^0 n$ → k = 0 → **Caso 2** → Θ(n log n). (ii) $f = n \log n = n^1 \log^1 n$ → k = 1 → Θ($n \log^{1+1} n$) = **$\Theta(n \log^2 n)$**.

CASO 3 — gana el trabajo de la raíz	
Cuándo se usa	El trabajo de combinar domina: f(n) es polinomialmente mayor que n^p y decrece al bajar de nivel (regularidad).
Cómo reconocerlo	$a < b^d$. Ejemplo: $2T(n/2) + n^2$, $T(n/2) + n$.
Ejemplo	$T(n) = 2T(n/2) + n^2$.
Pasos	a = 2, b = 2, $f = n^2$. p = 1. $n^2 = \Omega(n^{1+1})$ → ε = 1 → posible Caso 3. Regularidad: $a \cdot f(n/b) = 2 \cdot (n/2)^2 = n^2/2 \le (1/2) \cdot n^2$ → c = 1/2 < 1 ✓. Resultado: **$T(n) = \Theta(f(n)) = \Theta(n^2)$**.

6.2 Ejercicios resueltos

#	Recurrencia	a	b	$p = \log_b a$	f(n) vs n^p	Caso	Resultado
1	$T(n) = 9T(n/3) + n$	9	3	2	$n = O(n^{2-1})$	1	**$\Theta(n^2)$**
2	$T(n) = T(n/2) + 1$	1	2	0	$1 = n^0$ (k = 0)	2	**Θ(log n)**
3	$T(n) = 2T(n/2) + n$	2	2	1	$n = n^1$ (k = 0)	2	**Θ(n log n)**
4	$T(n) = 3T(n/4) + n \log n$	3	4	≈ 0.79	$n \log n = \Omega(n^{0.79+\varepsilon})$; reg.: $3(n/4)\log(n/4) \le \frac{3}{4} n \log n$ ✓	3	**Θ(n log n)**
5	$T(n) = 7T(n/2) + n^2$	7	2	≈ 2.81	$n^2 = O(n^{2.81-\varepsilon})$	1	**$\Theta(n^{\log_2 7}) \approx \Theta(n^{2.81})$**
6	$T(n) = 4T(n/2) + n^2 \log n$	4	2	2	$n^2 \log^1 n$ → k = 1	2	**$\Theta(n^2 \log^2 n)$**
7	$T(n) = 2T(n/2) + n^2$	2	2	1	$n^2 = \Omega(n^{1+1})$; reg.: $n^2/2 \le \frac{1}{2} n^2$ ✓	3	**$\Theta(n^2)$**
8	$T(n) = 4T(n/2) + n$	4	2	2	$n = O(n^{2-1})$	1	**$\Theta(n^2)$**
9	$T(n) = T(n/2) + n$	1	2	0	$n = \Omega(n^{0+1})$; reg.: $n/2 \le \frac{1}{2} n$ ✓	3	**Θ(n)**
10	$T(n) = 2T(n/2) + \sqrt{n}$	2	2	1	$\sqrt{n} = O(n^{1-1/2})$	1	**Θ(n)**

Contexto: #2 búsqueda binaria; #3 Merge Sort; #5 Strassen; #7 y #9 dominan las operaciones de la raíz. Todos verificados numéricamente evaluando T(n) para n grandes.

6.3 Cuándo NO se puede usar el Teorema Maestro

Situación	Ejemplo	Qué hacer
El subproblema no es n/b (reduce por resta)	$T(n) = T(n-1) + n$	Desenrollar: $n(n+1)/2$ → $\Theta(n^2)$
Reducción por resta con varias ramas	$T(n) = 2T(n-1) + 1$	Árbol / desenrollar → $\Theta(2^n)$
Subproblemas de distinto tamaño	$T(n) = T(n/3) + T(2n/3) + n$	Árbol de recursión o Akra–Bazzi → Θ(n log n)
a no constante o a < 1	$T(n) = 2^n T(n/2) + n^n$; $T(n) = \frac{1}{2} T(n/2) + n$	Otro método (sustitución, árbol)
f(n) no es polinomialmente comparable con n^p	$T(n) = 2T(n/2) + n/\log n$	Versión extendida: k = −1 → Θ(n log log n)
Caso 3 sin regularidad comprobable	f(n) oscilante, $a \cdot f(n/b) > c \cdot f(n)$	No se puede concluir con el teorema básico
f(n) negativa	$T(n) = 2T(n/2) - n$	Fuera de hipótesis
Reducción n → √n u otra no lineal	$T(n) = T(\sqrt{n}) + 1$	Cambio de variable m = log n → Θ(log log n)

Versión extendida (caso 2 con k cualquiera): $f(n) = \Theta(n^p \log^k n)$: si k > −1 → $\Theta(n^p \log^{k+1} n)$; si k = −1 → $\Theta(n^p \log \log n)$; si k < −1 → $\Theta(n^p)$.

7. Cómo mejorar algoritmos ineficientes

Cómo detectar que un algoritmo se puede mejorar

• Recursión que llama varias veces con los **mismos argumentos** → memoización / programación dinámica. • Búsquedas lineales repetidas → hash table, o ordenar una vez + búsqueda binaria. • Ciclos anidados que recalculan lo mismo → precomputar (sumas de prefijos), dos punteros. • Copias de arreglos (slicing, concatenar en ciclos) → trabajar con índices. • Entrada ordenada que no se aprovecha → búsqueda binaria / dos punteros.

7.1 Memoización

Qué es: guardar el resultado de una función para cada argumento ya calculado y reutilizarlo en lugar de recalcularlo. **Cómo funciona:** antes de calcular, se consulta un diccionario/arreglo (memo); si está, se devuelve; si no, se calcula, se guarda y se devuelve. **Conviene cuando** la función es pura (mismo argumento → mismo resultado) y el árbol de recursión repite subproblemas, con pocos argumentos distintos.

Original: Fibonacci ingenuo

```
def fib(n):
    if n <= 1:
        return n
    return fib(n - 1) + fib(n - 2)
```

Optimizado: con memoización

```
def fib_memo(n, memo=None):
    if memo is None: memo = {}
    if n <= 1: return n
    if n in memo: return memo[n]   # ya calculado
    memo[n] = fib_memo(n-1, memo) + fib_memo(n-2, memo)
    return memo[n]
```

Versión	Tiempo	Espacio	Por qué
Ingenuo	$T(n) = T(n-1) + T(n-2) + 1 = \Theta(\varphi^n) \subset O(2^n)$, $\varphi \approx 1.618$	O(n) (profundidad de la pila)	Recalcula los mismos fib(k) muchísimas veces. fib(40) ≈ $3.3 \cdot 10^8$ llamadas.
Memoización (top-down)	**O(n)**	O(n) memo + O(n) pila = **O(n)**	Hay n+1 subproblemas distintos; cada uno se expande una sola vez con trabajo O(1); las demás llamadas devuelven de inmediato. ≈ 2n llamadas (fib(40) ≈ 79).
Iterativo (bottom-up)	**O(n)**	**O(1)** con dos variables (O(n) si guarda tabla)	Calcula en orden fib(0), fib(1), ... sin recursión ni pila.

Se asume suma en O(1). Con enteros enormes, cada suma cuesta proporcional al nº de dígitos. Python limita la recursión (~1000): para n grande usa la versión iterativa.

Bottom-up con espacio constante

```
def fib_it(n):                # tiempo O(n), espacio O(1)
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a
```

7.2 Programación dinámica (PD)

Qué es: paradigma para resolver problemas que tienen **subproblemas superpuestos** (se repiten) y **subestructura óptima** (la solución se arma con soluciones de subproblemas), guardando cada resultado una vez. **Diferencia con memoización:** la memoización es una *técnica* (caché de resultados) aplicable a cualquier función pura; la PD es un *método de diseño* (estado + recurrencia + casos base + orden) que se implementa top-down (con memoización) o bottom-up (tabulación).

	Top-down (memoización)	Bottom-up (tabulación)
Dirección	De n hacia los casos base (recursión)	De los casos base hacia n (ciclos)
Calcula	Solo los subproblemas que se necesitan	Todos los subproblemas de la tabla
Ventajas	Se escribe directo desde la recurrencia	Sin recursión; permite reducir memoria (guardar solo la fila previa)
Riesgos	Pila O(profundidad); límite de recursión	Hay que acertar el orden de llenado

Receta de PD y cómo calcular su complejidad

1) Define el estado (qué significa dp[i][j]). **2)** Escribe la recurrencia. **3)** Casos base. **4)** Orden de cálculo. **5)** Respuesta. **6)** ¿Se puede reducir memoria?

Complejidad en tiempo = **nº de estados × costo de cada transición**. Úsala cuando los estados distintos sean polinomiales (ej.: mochila 0/1 O(n·W), LCS O(n·m), cambio de monedas).

Ejemplo bottom-up: sin PD la recursión sería exponencial (C(m+n−2, m−1) llamadas)

```python
def caminos(m, n):   # caminos esquina sup-izq → inf-der, mover ↓ o →
    dp = [[1] * n for _ in range(m)]          # estado: dp[i][j]
    for i in range(1, m):
        for j in range(1, n):
            dp[i][j] = dp[i-1][j] + dp[i][j-1]   # recurrencia
    return dp[m-1][n-1]       # tiempo O(m·n), espacio O(m·n) → O(n) con una fila
```

7.3 Optimización de Merge Sort

Original (con copias)

```python
def merge_sort(a):
    if len(a) <= 1: return a
    m = len(a) // 2
    L = merge_sort(a[:m])       # copia
    R = merge_sort(a[m:])       # copia
    return merge(L, R)          # lista nueva
```

Optimizado: índices + buffer reutilizable + insertion sort + chequeo

```python
CUTOFF = 16
def merge_sort_opt(a):
    aux = a[:]                  # UN buffer, se crea una vez
    _sort(a, aux, 0, len(a))

def _sort(a, aux, lo, hi):      # ordena a[lo:hi]
    if hi - lo <= CUTOFF:       # subarreglo pequeño
        insertion(a, lo, hi); return
    mid = (lo + hi) // 2
    _sort(a, aux, lo, mid); _sort(a, aux, mid, hi)
    if a[mid-1] <= a[mid]: return   # ya ordenado: no mezcla
    aux[lo:hi] = a[lo:hi]
    i, j = lo, mid
    for k in range(lo, hi):
        if i >= mid:        a[k] = aux[j]; j += 1
        elif j >= hi:       a[k] = aux[i]; i += 1
        elif aux[j] < aux[i]: a[k] = aux[j]; j += 1
        else:               a[k] = aux[i]; i += 1

def insertion(a, lo, hi):
    for i in range(lo + 1, hi):
        x = a[i]; j = i - 1
        while j >= lo and a[j] > x:
            a[j + 1] = a[j]; j -= 1
        a[j + 1] = x
```

Optimización	Qué hace	Efecto
Evitar copias (índices lo/hi en lugar de a[:m])	No crea sublistas nuevas en cada llamada	Menos memoria asignada y menos tiempo en copiar (constante)
Buffer auxiliar reutilizable	Un solo arreglo aux de tamaño n para todas las mezclas	Espacio auxiliar O(n) fijo, sin asignaciones repetidas
Insertion sort en subarreglos pequeños (≤ 16)	Evita el costo de recursión en bloques chicos	Mejora la constante; con cutoff constante el orden no cambia
Saltar la mezcla si a[mid−1] ≤ a[mid]	Segmentos ya ordenados no se mezclan	Entrada ya ordenada: O(n) en lugar de O(n log n)

	Merge Sort original	Merge Sort optimizado
Tiempo peor caso / promedio	Θ(n log n)	Θ(n log n) ← **el orden asintótico NO cambia**
Mejor caso (ya ordenado)	Θ(n log n)	O(n) por el chequeo (solo ese caso)
Espacio auxiliar	O(n) (picos con copias; ≈ n log n elementos asignados en total)	O(n) un buffer + O(log n) de pila
Constantes / rendimiento práctico	Altas (muchas asignaciones)	Menores: menos copias, menos llamadas

No confundir

Estas mejoras son de **constantes, memoria o rendimiento práctico**. El orden asintótico del peor caso sigue siendo O(n log n): para bajar de ahí haría falta otro enfoque (no existe con comparaciones: cota Ω(n log n)).

7.4 Otras técnicas y sus condiciones

Problema	Técnica	Beneficio	Condiciones y límites
Cálculos repetidos	Memoización	Evita recomputaciones	Función pura; gasta memoria O(nº de estados)
Subproblemas superpuestos	Programación dinámica	Reutiliza resultados	Requiere subestructura óptima y pocos estados distintos
Búsqueda lineal en datos ordenados	Búsqueda binaria	Reduce a O(log n)	Datos ordenados y acceso por índice; ordenar cuesta O(n log n)
Bucles innecesarios	Reducción de recorridos	Menos operaciones	Fusionar ciclos solo cambia constantes; baja el orden si elimina un ciclo anidado (prefijos, dos punteros)
Estructuras inadecuadas	Hash tables	Búsquedas promedio O(1)	Peor caso O(n) por colisiones; O(n) memoria; sin orden. Ej.: par con suma S de $O(n^2)$ a O(n)
Ordenamiento ineficiente	Divide y vencerás	Puede lograr O(n log n)	Problema divisible y combinar barato; costo de recursión y memoria extra
Consultas de rango repetidas	Sumas de prefijos	Consulta O(1) tras preprocesar O(n)	Datos que no cambian (o cambian poco)

1 · TABLA DE COMPLEJIDADES

O(·)	Patrón típico	Ejemplo
1	acceso directo	a[i]
log log n	i = i*i ; n = √n	interpolation search
log n	i *= 2 ; i //= 2	búsqueda binaria
√n	while i*i ≤ n	primalidad
n	un recorrido	buscar / sumar
n log n	n × log n ; dividir + recorrer	Merge / Heap Sort
n^2	dos ciclos	pares, burbuja
n^3	tres ciclos	matrices ingenuo
2^n	ramificación recursiva	fib ingenuo
n!	permutaciones	fuerza bruta

$1 < \log\log n < \log n < \sqrt{n} < n < n\log n < n^2 < n^3 < 2^n < n! < n^n$

n = 2^{32}: log n = 32 · log log n = 5 · n log n ≈ $1.4\cdot10^{11}$

2 · REGLAS DE CICLOS

Código	Iteraciones	Total
i += c	n/c	**n**
i *= c (c ≥ 2) / i /= c	$\log_c n$	**log n**
i = i*i ($i_0 \geq 2$)	log log n	**log log n**
i*i ≤ n / i += k; k++	√n	**√n**
anidados independientes	multiplicar	**n·n = n^2**
j < i (dependiente)	n(n−1)/2	**n^2**
for n { while j *= 2 }	n · log n	**n log n**
j en range(0, n, i)	$\Sigma\, n/i = n\cdot H_n$	**n log n**
for j < i; i *= 2	suma geométrica ≈ 2n	**n**
bloques seguidos	sumar	**máx**
if / else	cond. + mayor rama	**peor caso**

3 · REGLAS DE RECURSIVIDAD

Recurrencia	Resultado
T(n−1) + 1	**O(n)**
T(n−1) + n	**$O(n^2)$**
T(n/2) + 1	**O(log n)**
T(n/2) + n	**O(n)**
2T(n/2) + 1	**O(n)**
2T(n/2) + n	**O(n log n)**
4T(n/2) + n	**$O(n^2)$**
3T(n/2) + n	**$O(n^{1.58})$**
2T(n−1) + 1	**$O(2^n)$**
T(n−1) + T(n−2)	**$\Theta(\varphi^n) \subset O(2^n)$**

Cuenta: nº de llamadas × trabajo por llamada, o profundidad × trabajo por nivel. Sin n/b → desenrollar / árbol.

4 · LOGARITMOS

- $\log_b x = y \Leftrightarrow b^y = x$; log(xy) = log x + log y ; log(x/y) = log x − log y
- $\log(x^k) = k\cdot\log x$; cambio de base: $\log_b x = \log_c x / \log_c b$ (constante)
- $b^{\log_b x} = x$; $a^{\log_b c} = c^{\log_b a}$; log(n!) = Θ(n log n)
- $\Sigma\, i = n(n+1)/2$; $\Sigma\, 2^i = 2^{k+1} - 1$; $\Sigma\, 1/i \approx \ln n$
- $2^{2^k} = n \Rightarrow k = \log\log n$; $n^{1/2^k} = 2 \Rightarrow k = \log\log n$

5 · TEOREMA MAESTRO

$T(n) = a\cdot T(n/b) + f(n)$ $a \geq 1$, $b > 1$

a = nº subproblemas · b = reducción · f(n) = trabajo fuera de las llamadas · **$p = \log_b a$** · k = exponente del log

Caso	Si f(n) …	T(n)
1	$= O(n^{p-\varepsilon})$ (menor)	**$\Theta(n^p)$**
2	$= \Theta(n^p \log^k n)$, $k \geq 0$	**$\Theta(n^p \log^{k+1} n)$**
3	$= \Omega(n^{p+\varepsilon})$ y $a\cdot f(n/b) \leq c\cdot f(n)$, $c < 1$	**Θ(f(n))**

Atajo $f = n^d$: $a > b^d$ → caso 1 · $a = b^d$ → caso 2 · $a < b^d$ → caso 3.

Pasos: ① a ② b ③ f(n) ④ $p = \log_b a$ ⑤ compara f(n) con n^p ⑥ caso (en el 3 verifica regularidad) ⑦ Θ(…)

NO aplica: T(n−1) · a < 1 o variable · subproblemas desiguales · f = n/log n · caso 3 sin regularidad · f negativa · T(√n).

6 · ANALIZAR TIEMPO Y ESPACIO

- **Tiempo:** ① define n ② operación dominante ③ iteraciones de cada ciclo (despeja k) ④ anidado = ×, seguido = + ⑤ recursión: plantea T(n) y resuelve (Maestro / árbol / desenrollar) ⑥ simplifica; indica mejor / promedio / peor.
- **Espacio:** variables O(1) + estructuras (arreglo n → O(n), matriz n×m → O(nm)) + **pila = profundidad × marco**. Total = entrada + auxiliar.
- Merge Sort O(n) aux · Quick Sort O(log n) prom. · Heap Sort O(1) · fib ingenuo O(n) · fib memo O(n) · fib iterativo O(1) · bin. search rec. O(log n).

7 · TÉCNICAS DE OPTIMIZACIÓN

Problema	Técnica → beneficio
Cálculos repetidos	Memoización: fib 2^n → **O(n)**
Subproblemas superpuestos	PD: estados × costo transición
Búsqueda en datos ordenados	Binaria → **O(log n)**
Búsquedas repetidas	Hash → O(1) prom. (peor O(n))
Ciclos anidados redundantes	Prefijos / dos punteros → O(n)
Orden $O(n^2)$	Divide y vencerás → O(n log n)
Merge Sort lento	Índices, buffer único, insertion ≤16, saltar mezcla: **mejora constantes, NO el O(n log n)**

8 · ERRORES FRECUENTES

- Creer que O = peor caso: son cotas, no escenarios.
- Sumar cuando se multiplica (o al revés); olvidar que j < i da $n^2/2 \rightarrow O(n^2)$.
- i = i*i es **log log n**, no log n. i *= 2 con i = 0 → infinito.
- Fib ingenuo: tiempo $O(2^n)$ pero **espacio O(n)**. Pila = profundidad, no nº de llamadas.
- a[i:] / concatenar en recursión copia O(n) cada vez → $O(n^2)$.
- Maestro: olvidar regularidad (caso 3); en caso 2 el log sube a **k+1**; usarlo con T(n−1).
- Base del log no importa; la base en $p = \log_b$ a sí.
- $2^{n+1} = \Theta(2^n)$ pero $2^{2n} = 4^n \neq \Theta(2^n)$; $(\log n)^2 \neq \log n$; $\log(n^2) = 2 \log n$.
- Mezclar variables distintas: O(n + m), O(n·m); no fusionar.
- Memoizar o optimizar Merge Sort no cambia el orden asintótico del peor caso.
- Memo en Python: no uses lista/dict como argumento por defecto mutable (usa None).
- Si el ciclo tiene break/return, indica el caso (mejor / peor).

9 · MAESTRO: EJEMPLOS RÁPIDOS

Recurrencia	p	Caso	Θ
2T(n/2) + n	1	2	**n log n**
4T(n/2) + n	2	1	**n^2**
2T(n/2) + n^2	1	3	**n^2**
T(n/2) + 1	0	2	**log n**
3T(n/4) + n log n	0.79	3	**n log n**
7T(n/2) + n^2	2.81	1	**$n^{2.81}$**
4T(n/2) + n^2 log n	2	2	**$n^2 \log^2 n$**

10 · ALGORITMOS COMUNES

Algoritmo	Tiempo	Esp. aux.
Burbuja / Inserción	n^2 (inserción mejor: n)	O(1)
Merge Sort	n log n	O(n)
Quick Sort	n log n prom. · n^2 peor	O(log n)
Heap Sort	n log n	O(1)
Búsqueda binaria	log n	O(1)
Fibonacci memo / iter.	n	O(n) / O(1)